CortexHR: Resume Scoring Reliability

Four scoring bugs found and fixed, testable scoring functions, 21 tests and an evaluation harness for the LLM resume score.

Kriti Singhal · Technical documentation · October 2026

Project: github.com/Ashu-Stark/CortexHR-X-Tara

Contents

1. Summary

2. Where scoring sits

3. Bugs found and fixed

4. What changed

5. Tests

6. Evaluation design

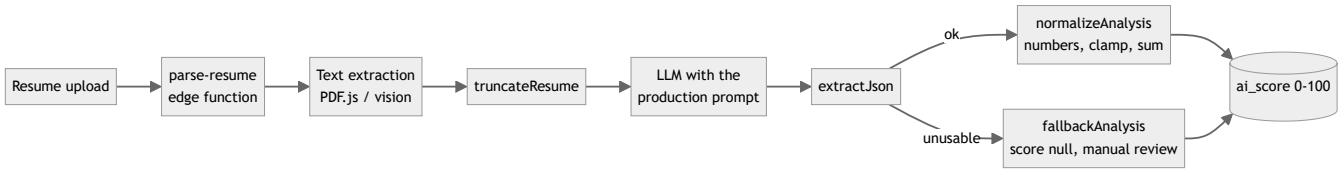
7. Interview questions

1 Summary

CortexHR is a team-built AI hiring platform: candidates upload resumes and an LLM scores each one against the job. This contribution (branch `kriti/scoring-reliability`) makes that score **trustworthy**: it finds and fixes four ways a model reply could produce a wrong or fake score, moves the scoring logic into testable functions, adds 21 tests and an evaluation harness.

4	21	0–100	~190
scoring bugs reproduced and fixed	unit tests, no API key needed	score always valid for the database	lines removed from the edge function

2 Where scoring sits



The scoring steps (`truncateResume`, `extractJson`, `normalizeAnalysis`, `fallbackAnalysis`) are new pure functions in `supabase/functions/_shared/scoring.ts`, so they run in Node tests without Deno, Supabase or a model.

3 Bugs found and fixed

Each was reproduced against the previous code before it was fixed.

#	Problem	Before	After
1	Model returns points as strings (`"points": "30"`)	`(a || 0) + (b || 0)` concatenated: total became `"302012874"`	Points converted to numbers first
2	A category above its maximum (e.g. 60/35)	Total 145; the update failed the database rule `ai_score BETWEEN 0 AND 100`	Each category clamped to its maximum; total always 0–100
3	Text around the JSON (`Here is the analysis: {…}`)	`JSON.parse` failed and the candidate got a "neutral" **50**	The JSON object is extracted from the reply
4	Any unusable reply	A fake 50 ranked an unparsed resume above genuinely weak candidates	`score: null` + `needs_manual_review: true`; the UI already shows no score for null

Also: the total is always the sum of the categories and a whole number (the column is `INTEGER`), and the raw model reply is no longer logged, because it contains candidates' personal data.

4 What changed

File	Change
`_shared/scoring.ts`	New: `extractJson`, `normalizeAnalysis`, `fallbackAnalysis`, `truncateResume`
`_shared/scoring-prompt.ts`	The scoring prompts moved **verbatim** out of the edge function, checked byte-for-byte against the old inline template, so the evaluation uses exactly the production prompt
`parse-resume/index.ts`	Uses the two modules; about 190 lines shorter
`tests/*.test.ts`	21 unit tests with Node's built-in test runner
`scripts/eval-scoring.ts`, `eval-data.json`	Scoring evaluation

5 Tests

Examples of what the 21 tests pin down:

- The score is always the sum of the categories, even when the model's own total is wrong.
- String, negative and missing points are handled (converted, or counted as 0).
- The result always fits the database rule: an integer between 0 and 100.
- A reply without a breakdown is an error, not a silent zero.
- The fallback marks the resume for manual review instead of inventing 50.
- Resume text is capped before it is sent to the model.

```
npm test                              # 21 tests, Node 22.6+, no install or API key
$env:GROQ_API_KEY = "<your key>"      # never commit it
npm run eval:scoring                  # ~30 API calls
```

6 Evaluation design

15 fictional resumes for 3 internship roles (backend, frontend, data), each labelled strong / medium / weak for its role. Each is scored twice with the production prompt, model (`llama-3.3-70b-versatile`) and temperature 0.

Question	Measure
Ranking	For each pair for the same role with different labels (24 pairs), is the stronger resume scored higher?
Consistency	Does the same resume get the same score twice?
Reliability	How many replies needed corrections or went to manual review?

> The evaluation has **not been run yet** (it needs an API key), so no results are claimed here. Run `npm run eval:scoring` and add the numbers from `scripts/eval-results.json`. Limits: resumes and labels were written by one person and the set is small. It checks that scoring is sensible and stable, not that it is fair across groups of people, which needs a much larger, carefully designed test.

7 Interview questions

Why was a fallback score of 50 harmful?

It ranked resumes the system failed to read above genuinely weak candidates. Missing data must look missing (null, manual review), not average.

Why move the prompt out verbatim and check it byte-for-byte?

An evaluation is only meaningful if it uses the exact production prompt; any accidental edit would test a different system.

Why clamp each category instead of only the total?

Clamping only the total hides which part was wrong and lets one category crowd out the others; per-category limits keep the breakdown meaningful.

Why not log the raw model reply?

It repeats details from the resume, which is personal data; logs are widely accessible and kept for a long time.